

Table of Contents

1. What is Git.....	1
Users.....	2
2. Github.....	3
Git and GitHub.....	3
3. Where Do We Start?	4
Installing Git on Windows machine.....	4
Installing Git on MAC machine.....	5
4. Bash Commands.....	6
5. Git workflow	7
6. Initialize new repository/Create a local repository.....	8
7. Establish connection between local machine with Github.....	9
8. How do I get my code on Github	11
Option 1.....	11
Option 2 - Git clone.....	12
9. Main Vs master branch.....	14
10. Branch-Rebase-PullRequest-Merge and other Git commands.....	15
11. Optional Git-Kraken GUI's	15

1. What is Git

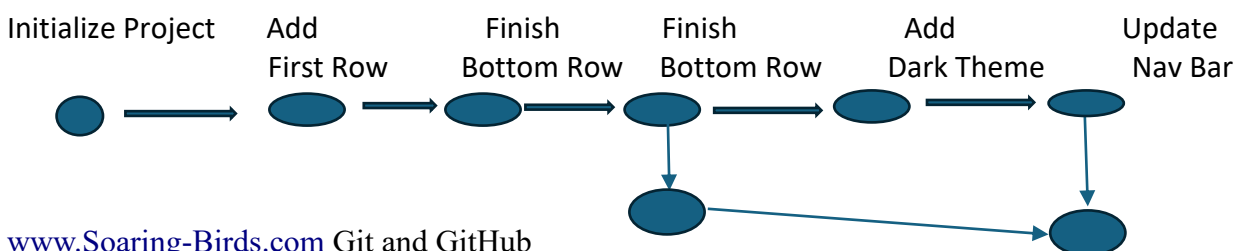
Git The world's most popular version control system. Version Control Version control is software that tracks and manages changes to files over time. Version control systems generally allow users to revisit earlier versions of the files, compare changes between versions, undo changes, and a whole lot more.

What exactly does Git do for us?

1. Track changes across multiple files
2. Compare versions of a project
3. "Time travel" back to old versions
4. Collaborate and
5. share changes
6. Combine changes
7. Git helps us...
8. Revert to a previous version

Example:

Assume you start a New Project. Each step developer of website saves their work and gives name. Since previous version is saved, user can anytime “Time travel” meaning go to any saved changed. However, if other team members are working on other feature like designing home pages or adding footers. Their work can be combined later.



Other Version control rules

Git is just one VCS Git is just one of the many version control systems available today. Other well-known ones include Subversion, CVS, and Mercurial. They all have similar goals but vary significantly in how they achieve those goals. Fortunately, we only need to care about Git because Git is the clear "winner". According to survey nearly 90% of respondents reported Git as their version control system of choice. Over the last few years, the survey hasn't even bothered to ask about version control because Git is so widely used.

Git	– 88.4%
Sub version	- 16.6 %
Team foundation Version Control	– 11.3%
Copying and pasting files to network shares	– 7.7%
Zip file back-ups	– 7.7%
Mercurial	– 3.7%

Linus Torvalds is a legendary software engineer. He is the creator and main developer behind Linux and Git! In 2005, while working on Linux, he became frustrated with the available version control systems. The existing tools were slow, closed source, and usually paid. Torvalds wanted a version control system that was super-fast AND free, unlike the existing tools. On April 3rd, 2005, he got to work on his own VCS, which would become Git. In a matter of days, he had most basic functionality done. The first official Git release came a couple months later. 15 years later in 2020, over 90% of developers worldwide use Git daily.

Users

Engineers & Coders: From massive tech giants like Facebook and Google to the tiniest startups, developers across the globe use Git. If you plan on becoming a developer, Git is essentially a must-have.

Tech-Adjacent Roles: Many people in non-developer roles end up learning the basics of Git to collaborate with their coworkers. Designers often need to work with Git.

Governments: In recent years, governments have started using Git to manage the drafting of laws. Washington DC City council uses Git & Github to publish its laws. Citizens have even found and fixed typos collaboratively using Git!

Scientists: Git is commonly used by research teams at universities and agencies around the world to manage code, data sets, and more. Git + Github is especially powerful in the peer-reviewed world of science and research.

Writers: Some writers love using Git to manage drafts of complex novels, screenplays, or other works with lots of moving parts and constant changes across multiple files. Git is gaining popularity for use in collaborative textbook writing with multiple authors.

Anyone Really: People find very creative uses for Git ranging from keeping a daily diary to drafting PhD theses to tracking changes to photoshop files. At least one composer writes his symphonies using Git!

2. Github

Github is a hosting platform for Git repositories. You can put your own Git repos on Github and access them from anywhere and share them with people around the world.

Beyond hosting repos, Github also provides additional collaboration features that are not native to git. (but super useful). Basically, Github helps people share and collaborate on repos.

GitHub gives your code a home in the cloud. Where can you record and rewind any change to your code. Host it all for free with unlimited public and private repositories.

Today Github is the home of open-source projects on the internet.

Your Github profile showcases your own projects and contributions to others' projects. It can act as a sort of resume that many employers will consult in the hiring process. Additionally, you can gain some clout on the platform for creating or contributing to popular projects.

Github is not only options, others like GitLab, Gerrit, Bitbucket do the same. GitHub was founded in 2008; GitHub is now the world's largest source code. In early 2020, GitHub reported having over 40 million users and over 190 million repositories on the platform.

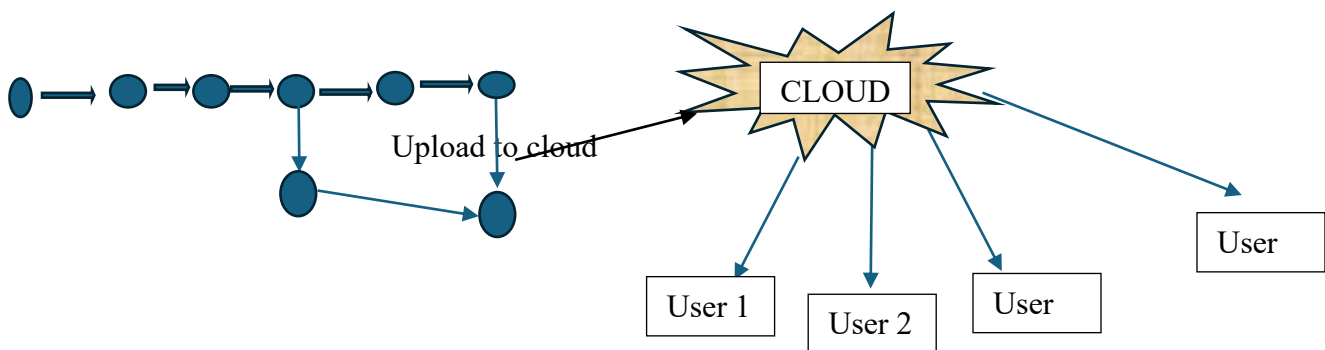
Github offers its basic services for free. While Github does offer paid Team and Enterprise tiers, the basic Free tier allows for unlimited public and private repos and collaborations.

www.github.com/pricing

Git and GitHub

<u>Git</u>	<u>Github</u>
Git is the version control software that runs locally on your machine. You don't need to register for an account. You don't need the internet to use it. You can use Git without ever touching Github	Github is a service that hosts Git repositories in the cloud and makes it easier to collaborate with other people. You do need to sign up for an account to use Github. It's an online place to share work that is done using Git.

From the previous example, all these are all on local machines. What if the local machine is lost or goes bad. All work is lost. I can upload my history to cloud and recover from the cloud. Using Github does the same job. The other advantage is that user/owner of work can share with others in the team as well.



3. Where Do We Start?

First, we will start with Git and setting local copy of repo/code. Then register account in GitHub and push our repo.

Git Is (Primarily) A Terminal Tool Git was created as command-line tool. To use it, we run various git commands in a Unix shell. This is not the most user-friendly experience, but it's at the very core of Git!

Below are the steps to set up Git and GitHub

- 1> Install Git
- 2> Create a local repository
- 3> Connect to the remote repository
- 4> Push the file to GitHub
- 5> Basic Git commands

Installing Git on Windows machine

Installing Git locally depends on your **specific operating system**. Git is intended to run on Unix-style systems like Linux and MacOS, so if you're on a Windows machine you may need to take a couple extra steps.

Step 1: Download and install Git Bash and Git

Windows comes with a default command line interface called command Prompt that is not Unix-based. Bash is the command line interface that is default in Linux and Mac. Git was designed to run on Bash. **Git Bash** emulates the unix-based Git command-line experience for Windows machines, and it's super easy to install!..

Follow the Git homepage link to install on operating systems.

<https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>. **Quick steps are captured here**

The most official release is available for download on the Git website. Just go to <https://git-scm.com/download/win> and the download. Note that this is a project called Git for Windows, not standalone Git itself; for more information on it, go to <https://gitforwindows.org>.

Install the downloaded .exe file. While installing you can either select default editor as vim or visual Studios. If selected Visual Studios, install VC first and resume Git installation.

For all other options, select the default as guided by installer. Example like

- 1> Initial branch on repository - Select "Let Git decide"
- 2> Terminal to use on GitBash – Select "MinTTY" (Default by installer)

After Installing completes, select "Launch GitBash" and "Finish". Hopefully you will get terminal as

<user_name> MINGw64 ~ \$

If you close this Window, it can reopen by searching "GitBash" in the Window's program menu. GitBash is the name of the application which had Git and Bash.

Once install is complete check the installed version with this command

➤ **git --version**

Step 2: Configure Git

Now that Git is hopefully installed, it's time to configure some basic information like Your name and Your email. If you are using a GUI, it should prompt you for your name and email the first time you open the app.

To configure the name and email that Git will associate with your work, run this command:

Navigate to Programs -> Git Bash -> You need to execute the commands below on this terminal.

- `git config --global user.name "YOUR_NAME(in quotes)"`
- `git config --global user.email blah@blah.com`

When we get to Github, you'll want your Git email address to match your

For any help, use the following command **\$ git help config or \$ git config --help**. This command will lead you to a browser of config commands.

Set values can be verified using the commands below.

- `git config user.name`
- `git config user.email`

Installing Git on MAC machine

First, check to see if you have Git installed already using the command **git --version**

If not, or if you have an old version, download the latest Git installer package using the link in the description

Verify your install worked by running `git --version` again afterwards.

Command Line Pros	Command Line Cons
• Git is a command-line tool. All the documentation and resources online will refer to the command-line. • The command-line can be way faster once you get comfortable with it! • Some of the more advanced Git features are only available on the command-line. • The commands are always the same, no matter what machine you are on!	• Not beginner friendly. At All. Can be difficult to learn and remember the commands at first. • Even for some practiced users, the command line interface is just not a good experience. It's really a matter of preference.

4. Bash Commands

These commands are used in Git bash terminal to navigate in the Git repo. So, practice these commands as these are very necessary.

<u>Command description</u>	<u>Commands</u>
Print the lists the files in the directory. -al is the option for ls command which asks to show all files including hidden folders. Hidden folder starts with “.Folder/file name”	\$ ls \$ start . (open . in MAC) \$ ls -al <Full path of folderName>
Print Present directory – complete path. ~ called tilde is Home directory.	\$ pwd
Create a local directory using the following command:	\$ mkdir ANY_DIRECTORY_NAME
Navigate to the new directory that was created in last command	\$ cd ANY_DIRECTORY_NAME
Navigate to one directory back	\$ cd ..
Creating empty new file	\$ touch ANY_FILE_NAME
Remove file	\$ rm ANY_FILE_NAME
Remove directory and its contents	\$ rm -rf ANY_DIRECTORY_NAME
Clear the screen	\$ clear

Important Git Bash commands in snapshot.

File Management

```
# Create a new directory
  mkdir new-project
# Create a new file
  touch README.md
# Copy files
  cp original.txt copy.txt
  cp -r source-folder/ destination-folder/ // Copy directory
# Move or rename files
  mv oldname.txt newname.txt
  mv file.txt /path/to/destination/
# Delete files and directories
  rm unwanted.txt
  rm -rf old-directory/ // Be careful with this!
```

Directory Navigation

```
# Show current directory
  pwd
# List files and directories
  ls
  ls -la // Show hidden files and details
# Change directory
  cd project-folder
  cd .. // Go up one level
```

5. Git workflow

1. Navigate to the directory where you want to use Git

\$ mkdir test , \$cd test

2. Initialize new repository

Use git init to create a new git repository. \$ git init

3. Create feature branch

\$ git checkout -b feature-payment

4. Make changes in initialized folder and check status

\$ git status

5. Stage the changes

\$ git add . // add dot says that group all the changes done in current and nested project directory.

6. Create checkpoint

\$ git commit "your message for checkpoint reference."

7. Push changes to remote branch

\$ git push <remote> <branch>

Important Git local Repository Management Commands in snapshot.

Creating and Cloning Repositories

git init // Initialize a new repository or Setup Git in development (working) folder

git clone <https://github.com/username/repository.git> // Clone an existing repository

git clone <https://github.com/username/repository.git> custom-folder-name // Clone to a specific folder

Tracking Changes

git status //Check repository status

git add filename.txt // Add specific file

git add . // Add files to staging area (all changes in folder)

git add *.js // Add all JavaScript files

git add src/ // Add entire directory

git commit -m "Add user authentication feature" // Commit changes

git commit --amend -m "Updated message" // Amend the last commit

Viewing History

git log // View commit history

Important Git Branch Management Commands in snapshot.

Branching is where Git's power truly shines, allowing parallel development streams.

Branch Management

git branch // Local branches

git branch -r // Remote branches

git branch -a // All branches

git branch feature-login // Create a new branch

git checkout -b feature-payment # Create and switch to a new branch

git checkout main // Switch branches

git branch -m old-name new-name // Rename a branch

git branch -d feature-complete // Delete a branch

git branch -D feature-broken // Force delete

Syncing with Remotes

git push origin feature-branch // Upload local changes

git branch --set-upstream-to=origin/main main // Set up branch tracking

6. Initialize new repository/Create a local repository

When working with Git, people often mention Git repository, short form as repo. Earlier we installed Git, that doesn't mean we can use it. We need to manually tell machines when to use Git and in which directory. We need to create a Git repository in the directory where we need to use Git. So, Git starts saving history in the directory after Git repo is setup.

Note Git commands to be executed in **Git bash terminal**.

Use git init to create a new git repository. Before we can do anything git-related, we must initialize a repo first!. This is something you do once per project. Initialize the repo in the top-level folder containing your project. Git tracks the directory and **all nested directories**.

More information on Git init command can be found at www.git-scm.com/docs/git-init. As the documentation says ".git" folder is created when repo is initialized. Use \$ls -al command inside the initialized folder to see ".git" hidden file. Do not modify the content inside the folder. This folder saves history. If this folder is deleted, entry history of the project will also be deleted.

Warning: DO NOT INIT A REPO INSIDE OF A REPO!. Before running git init, use git status to verify that you are not currently inside of a repo. If you do end up making a repo inside of a repo, you can delete it and try again! To delete a repo, locate the associated ".git" (dot is used to tell its hidden folder) directory and delete it.

\$ git init

Git is Watching Us after init command. Try making a new file in your repo and then run git status again. You'll see that git noticed the file, but it is "untracked. gives information on the current status of a git repository and its contents. More information on Git init command can be found at www.git-scm.com/docs/git-status

\$ git status

Now that we have initialized our project folder as Git repo, how to use Git and work inside folder?

As we progress in our project, we can create a check point. Check point not only saves a single file but saves the entire project's snapshot at that moment. This adding checkpoint is done with set of 2 commands.

\$ git add .

\$ git commit "your message for checkpoint reference."

Log command – will show the history with your recent commit on the top.

\$ git log

Add the "demo" to the current directory. Instead of.(dot), we can selectively add file names to the group.	\$ git add demo.txt
Adding: We use the git add command to stage changes to be committed. It's a way of telling Git, "Please include this change in our next commit"	
Committing: Making a commit is like taking a snapshot of a git repository in time. When saving a file, we are saving the state of a single file. With Git, we can save multiple files and folders together.	\$ git commit -m "Adding demo file"
Git Commit: We use the git commit command to commit changes from the staging area. When making a commit, we need to provide a commit message that summarizes the changes and work snapshotted in the commit	If \$ git commit That is without message , opens text editor for enter commit message".
	\$ git log
Use git status constantly to verify what changes are staged and not staged	\$ git status

7. Establish connection between local machine with Github

Step 1: Register at Github	If you don't have an account yet, first register at www.github.com While registering use the same email that you configured in Git. Check your configured email in GitBash terminal using command <code>\$ git config user.email</code>
Step 2: Configuring SSH Keys	You need to be authenticated on Github to do certain operations, like pushing up code from your local machine. Your terminal will prompt you every single time for your Github email and password, unless... You generate and configure an SSH key! Once configured, you can connect to Github without having to supply your username/password.
	Before generating SSH key check if you have an existing key. 1. Open Git Bash 2. Enter <code>ls -al ~/.ssh</code> to see if existing SSH keys are present. 3. Check the directory listing to see if you already have a public SSH key. By default, the filenames of supported public keys for GitHub are one of the following. <code>id_rsa.pub</code> <code>id_ecdsa.pub</code> <code>id_ed25519.pub</code> Tip: If you receive an error that <code>~/.ssh</code> doesn't exist, you do not have an existing SSH key pair in the default location. You can create a new SSH key pair in the next step. For more detail refer: Checking for existing SSH keys - GitHub Docs
	If you don't have a supported public and private key pair, or don't wish to use any that are available, generate a new SSH key. For more detail refer: Generating a new SSH key and adding it to the ssh-agent - GitHub Docs 1. Open Git Bash 2. Paste the text below, replacing the email used in the example with your GitHub email address. This creates a new SSH key, using the provided email as a label. <code>eval "\$(ssh-agent -s)"</code> <<<code>your_email@example.com??</code> When you're prompted to "Enter a file in which to save the key", you can press Enter to accept the default file location. Please note that if you created SSH keys previously, ssh-keygen may ask you to rewrite another key, in which case we recommend creating a custom-named SSH key. To do so, type the default file location and replace <code>id_ALGORITHM</code> with your custom key name. <pre>> Enter file in which to save the key (/c/Users/YOU/.ssh/id_ALGORITHM):[Press enter]</pre> At the prompt, type a secure passphrase. For more information, see "Working with SSH key passphrases." Or just press enter. <pre>> Enter passphrase (empty for no passphrase): [Type a passphrase] > Enter same passphrase again: [Type passphrase again]</pre>
Step 3: Adding your SSH key to the ssh-agent	Either old key or new generated key, add the key to the ssh-agent. <pre>// # start the ssh-agent \$ eval "\$(ssh-agent -s)" \$ ssh-add ~/.ssh/id_ed25519</pre>
Step 4: Add the SSH public key to	After you generate an SSH key pair, you must add the public key to GitHub.com to enable SSH access for your account. You can access and write data in repositories on GitHub.com

GitHub account.	using SSH (Secure Shell Protocol). When you connect via SSH, you authenticate using a private key file on your local machine. For more information https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account 1. Copy the SSH public key to your clipboard. If your SSH public key file has a different name than the example code, modify the filename to match your current setup. When copying your key, don't add any newlines or whitespace. # command copies the contents of the id_ed25519.pub file to your clipboard <u>\$ clip < ~/.ssh/id_ed25519.pub</u> <u>OR \$ cat ~/.ssh/id_ed25519.pub clip</u> 2. In the upper-right corner of any page, click your profile photo, then click Settings. 3. In the "Access" section of the sidebar, click "SSH and GPG keys" 4. Click "New SSH key" or "Add SSH key". 5. In the "Title" field, add a descriptive label for the new key. For example, if you're using a personal laptop, you might call this key "Personal laptop". 6. Select the type of key, either authentication or signing. 7. In the "Key" field, paste your public key 8. Click Add SSH key. 9. If prompted, confirm access to your account on GitHub.
Step 5: Test connection	<u>\$ ssh -T git@github.com</u>

```

prash@LAPTOP-A7SH54IA MINGW64 ~
$ ssh-keygen -t ed25519 -C [redacted]@gmail.com
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c:/Users/prash/.ssh/id_ed25519):
Created directory '/c:/Users/prash/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c:/Users/prash/.ssh/id_ed25519
Your public key has been saved in /c:/Users/prash/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:2AZKZ6vuNkriAghw9I3SCD5xvneAaXW4H/WTQxkRX7s [redacted]@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|  o..o...+*|
|  . . +O++ +*|
|  . o =O.=O.. o.+|
|.. . + *O.....+O|
|+ . o S. ....Eo|
|o . . . . .|
|o . . . . .|
|oo .o|
|...+o.|
+-----[SHA256]-----+

prash@LAPTOP-A7SH54IA MINGW64 ~
$ eval "$(ssh-agent -s)"
Agent pid 2059

prash@LAPTOP-A7SH54IA MINGW64 ~
$ ssh-add ~/.ssh/id_ed25519
Identity added: /c:/Users/prash/.ssh/id_ed25519 ([redacted]@gmail.com)

prash@LAPTOP-A7SH54IA MINGW64 ~
$ cat < ~/.ssh/id_ed25519.pub | clip

prash@LAPTOP-A7SH54IA MINGW64 ~
$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.114.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi [redacted]! You've successfully authenticated, but GitHub does not provide shell access.

prash@LAPTOP-A7SH54IA MINGW64 ~
$ |

```

8. How do I get my code on Github

How do I get the local Git repo to GitHub. How do I connect them and How do I setup GitHub repo and what do I do on my machine to tell about GitHub repo. There are 2 different approaches to do this.

Option 1: Existing Repo

If you already have an existing repo locally that you want to get on GitHub.

- Create a new repo on GitHub.
- Connect your local repo (add a remote)
- Push up your changes to GitHub.

Option 2: Start From Scratch

If you haven't begun work on your local repo, you can...

- Create a brand-new repo on GitHub.
- Clone it down to your machine.
- Do some work locally.
- Push up your changes to GitHub.

Pushing: To get your own changes and Git history up on GitHub, we need to PUSH them up. The typical workflow looks like this:

- Make some changes locally.
- Add and commit to those changes.
- Repeat...
- Push new commits up to GitHub.

Option 1

<u>Option 1</u>	
Move the local Git repo to empty GitHub branch. Perform \$ git log lot of commits get displayed. All these history and code/work will be moved to GitHub.	
Step 1: Create a new repo on GitHub	1. Sign in on GitHub. 2. In the upper-right corner of page next to profile picture, click "+ -> New Repository." 3. In new page, enter the <u>Repository</u> name under "Create a new repository." 4. Enter "Description" 5. Mark it as public or private. If it's your website, mark it as private, otherwise anyone can make changes to the website. Now we have done empty repository on GitHub. Local repo doesn't know anything about it.
Step 2: Give a name to GitHub url and set that name in the local Git repo.	Before we can push anything up to GitHub, we need to tell Git about our remote repository on GitHub. We need to set up a "destination" to push up to. In Git, we refer to these "destinations" as remotes. Each remote is simply a URL where a hosted repository lives. To view any existing remotes for you repository, we can run git remote or git

	remote -v (verbose, for more info) <u>\$ git remote -V</u> This just displays a list of remotes. Since we have not yet set the remote, this command will not show anything. Adding A New Remote: A remote is really two things: a URL and a label. To add a new remote, we need to provide both to Git. <u>\$ git remote add <name> <url></u> Ex: \$ git remote add origin www.bhal.com //Tells Git anytime name "origin" is used, it's referring to this particular GitHub repo URL. Remember, by setting up a remote we are just telling Git about a remote repository URL. We have not "communicated" with the GitHub repo at all yet.
They are not commonly used, but there are commands to rename and delete remotes if needed. git remote rename <old> <new> git remote remove <name>	
Step 3: Push changes to remote GitHub	Now that we have a remote set up, let's push some work up to GitHub! To do this, we need to use the git push command. We need to specify the remote we want to push up to AND the specific local branch we want to push up to that remote. <u>\$ git push <remote> <branch></u> Ex: <u>\$ git push origin master</u> //tells git to push up the local-master branch to our origin remote. For the first time, there are no branches on remote. It creates a master branch. From next time, this command pushes local-branch to remote-master by default. But if we don't want to push to remote-master \$ git push origin <local-branch>: <remote-branch> \$ git switch -c cats \$ git push origin cats:master // This pushed cats to master branch
-u option allows us to set the upstream of the branch we are pushing. You can think of this as a link connecting our local branch to a branch on GitHub. Running <u>\$ git push -u origin master</u> sets the upstream of the local master branch so that it tracks the master branch on the origin repo. Local Git remembers it for future push commands. Now that it remembers, do some changes and commit. \$ git switch -c dogs \$ git push -u origin dogs \$ git touch dogs.txt \$ git commit -m "text push" \$ git push // changes are pushed to existing dogs branch, not to master.	
Step 4: Check GitHub repo	Check the recent commits with time reflecting on GitHub. Check "All commits" page.

Option 2 - Git clone

Connecting GitHub repo to local repo

So far, we've created our own Git repositories from scratch, but often we want to get a local copy of an existing repository instead. This opens the path for collaboration and exposure. Or you haven't started your work locally. You know you want to use GitHub. You can start creating empty repo on GitHub and clone it back to local machine. **This automatically configure remote.** Then do some work and push changes to GitHub repo.

To do this, we can clone a remote repository hosted on GitHub. All we need is a URL that we can tell Git to clone for use. To clone a repo, simply run `git clone <url>`. Git will retrieve all the files associated with the repository and will copy them to your local machine. In addition, Git initializes a new repository on your machine, giving you access to the full Git history of the cloned project. Make sure you are not inside of a repo when you clone!. For more information refer : www.git-scm.com/docs/clone

\$ git clone https://github.com/...

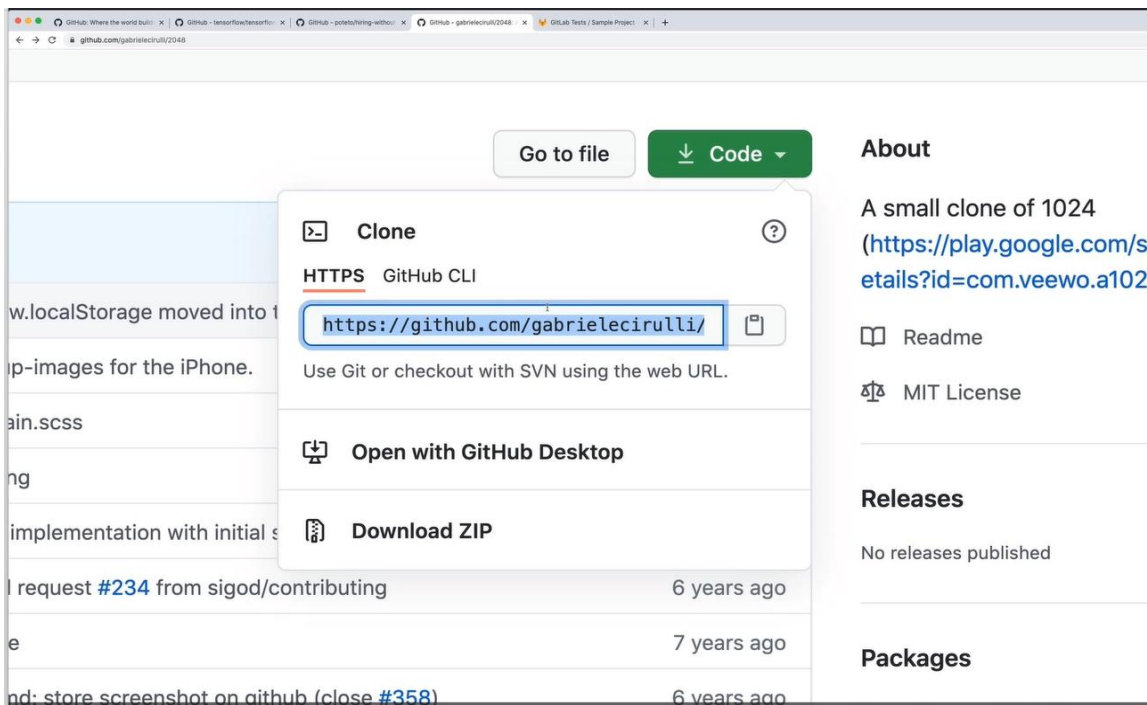
// Do some changes.

\$ git diff

\$ git push origin master

Anyone can clone a repository from GitHub, **provided the repo is public**. You do not need to be an owner or collaborator to clone the repo locally to your machine. You just need the URL from GitHub. You need permission to push up your own changes to the GitHub repo.

url for cloning can be copied from project path as shown below.



9. Main Vs master branch

In 2020 GitHub started renaming the default branch from master to main. While creating branch new repo on GitHub, if you select “Add a README File”, notice the last line. It says, “main” as default branch. That is instead of creating empty repo, if repo is created with ReadMe file. It automatically wants to add some branch because you cannot commit any file without any branch. We would always change that it always defaults to master in “settings” (lined at the end of the line as shown in picture). Or else go to “settings -> branches -> change default branch “

Owner * Repository name *

Colt / colors ✓

Great repository names are short and memorable. Need inspiration? How about shiny-doodle?

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

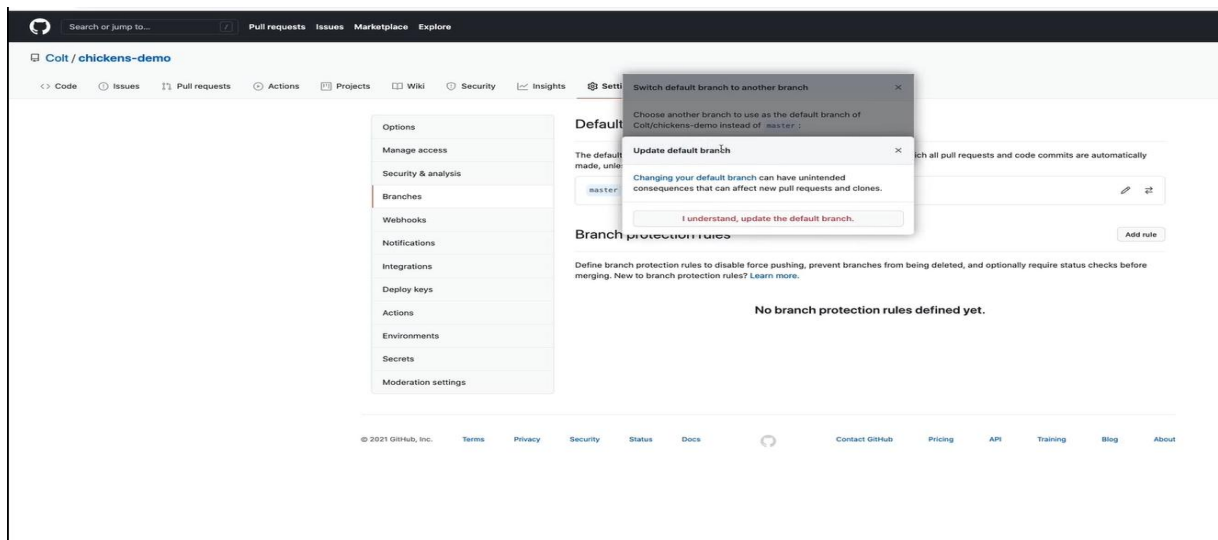
☒ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

☐ **Add .gitignore**
Choose which files not to track from a list of templates. [Learn more.](#)

☐ **Choose a license**
A license tells others what they can and can't do with your code. [Learn more.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

Create repository



You can also rename master to main. There is no significance to branch name.

\$ git branch -M main

10. Branch-Rebase-PullRequest-Merge and other Git commands

Remember GitHub has advantages for collaboration. When more than one person is working on project other commands like Rebase, Merger, pull request come to use.

Collaboration	If you ever plan on working on a project with at least one other person, GitHub will make your life easier! Whether you're building a hobby project with your friend or collaborating with the entire world, GitHub is essential!
Exposure	Your GitHub profile showcases your own projects and contributions to others' projects. It can act as a sort of resumé that many employers will consult in the hiring process. Additionally, you can gain some clout on the platform for creating or contributing to popular projects.
Stay Up to Date	Being active on GitHub is the best way to stay up to date with the projects and tools you rely on. Learn about upcoming changes and the decisions/debate behind them.

11. Optional Git-Kraken GUI's

Over the last few years, companies have created graphical user interfaces for Git that allow people to use Git without having to be a command-line expert.

Popular Git GUI's include.

- Github Desktop
- SourceTree
- Tower
- Gi Kraken
- Ungit

GUI Clients Pros	GUI Clients Cons
• Way lower barrier-of-entry for beginners compared to the command-line. • Friendlier to use. Can be much better. experience (when it works) • Some people prefer the visual experience, even those who can use the command-line	• At times, there is lots of "magic" involved. The inner workings of Git are obfuscated and hidden away with GUI's. • Often leads to dependance on a particular piece of software. • When things go seriously wrong, it can be very challenging to fix without the command-line. • The interfaces, buttons, and menus vary. between different GUI's.